HIREMATE: Hierarchical Approach for Efficient Re-materialization of Neural Networks

Julia Gusak [*1] **Xunyi Zhao** [*1] **Théotime Le Hellard** [*2] **Zhe Li** [1] **Lionel Eyraud-Dubois** [1] **Olivier Beaumont** [1]

Abstract

Training deep neural networks (DNNs) on memory-limited GPUs is challenging, as storing intermediate activations often exceeds available memory. Re-materialization, a technique that preserves exact computations, addresses this by selectively recomputing activations instead of storing them. However, existing methods either fail to scale, lack generality, or introduce excessive execution overhead. We introduce HIREMATE, a *hierarchical* re-materialization framework that recursively partitions large computation graphs, applies optimized solvers at multiple levels, and merges solutions into a global efficient training schedule. This enables scalability to significantly larger graphs than prior ILP-based methods while keeping runtime overhead low. Designed for single-GPU models and activation re-materialization, HiRemate extends the feasibility of training networks with thousands of graph nodes, surpassing prior methods in both efficiency and scalability. Experiments on various types of networks yield up to 50-70% memory reduction with only 10-15% overhead, closely matching optimal solutions while significantly reducing solver time. Seamlessly integrating with PyTorch Autograd, HiRemate requires almost no code change to use, enabling broad adoption in memory-constrained deep learning.

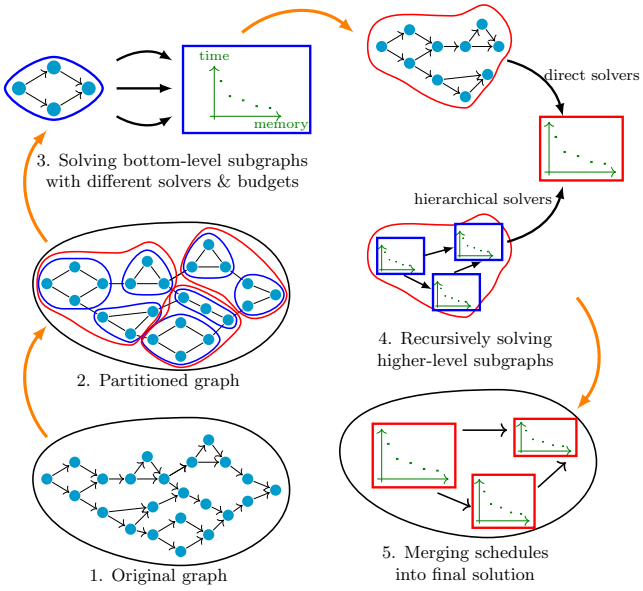

Figure 1. **HIREMATE** takes a PyTorch `nn.Module` and creates a `nn.Module`, which provides same outputs while satisfying peak memory budget constraints B during training. (1) Obtain data-flow graph G from the PyTorch module. (2) Recursively partition G with **H-Partition** (Section 3.1) into small-size subgraphs. (3)-(5) Obtain a re-materialization schedule from G with budget B using **H-Solver** (Section 3.2). (6) Produce a new `nn.Module` whose execution follows the schedule (Appendix).

1. Introduction

Modern Neural Networks (NN) undergo several important evolutions which have consequences on the computation and memory requirements, from the first vision networks like ResNet-50 (Wu et al., 2019) to Natural Language Processing transformer-based models (Vaswani et al., 2017) like GPT. On the one hand, the size of the models and the resolution of the data are increasing, which raises problems for the storage of both weights and activations. On the other hand, the first models had chain-like structures (sequence of convolution layers) and then moved to chains of complex blocks (chains of Transformer blocks for GPT-like models). Finally, recent NN exhibit arbitrarily complex dependency graph structures between layers of the neural network: for example, UNO (Wen et al., 2022) is structured as a U-Net of complex blocks, and encoder-decoder transformers (Vaswani et al., 2017) feature very long skip connections.

[*]Equal contribution [1]Inria Center at the University of Bordeaux [2]École Normale Supérieure, PSL University, Paris. Correspondence to: Julia Gusak <yulia.gusak@inria.fr>, Lionel Eyraud-Dubois <lionel.eyraud-dubois@inria.fr>.

Proceedings of the 42^{nd} International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025.

Re-materialization is a well known and efficient technique to limit the memory requirements related to activations during training. The idea is to avoid storing all the necessary activations because of the subsequent dependencies during the Forward phase and the Backward phase (those related to the Stochastic Gradient Descent mechanism). Some activations are computed and then deleted to make room in memory, and they will have to be re-computed later when needed. In the case of simple chains (Beaumont et al., 2019) and in the case of chains of complex blocks (Zhao et al., 2023), re-materialization has shown its efficiency: it is often possible to save 50% of memory for a computational overhead of about 10 to 15%.

From a theoretical point of view, the problem is to minimize the computational time required to execute the forward and backward passes under a predefined memory budget. It has been proven NP-Hard in (Naumann, 2008) for the case of general dependency graphs represented as general data-flow graphs. Some solutions such as TW-REMAT (Kumar et al., 2019) or CHECKMATE (Jain et al., 2020) have nevertheless been designed to deal with the case of general graphs, but both suffer from a number of limitations. These limitations stem either from the computational cost and the scalability of the algorithm that generates the re-materialization strategy, or from the overhead of that re-materialization strategy during the training phase, as discussed in Section 2. Another line of research is to propose re-materialization strategies whose computational cost and overhead are controlled, as in ROTOR (Beaumont et al., 2019) and ROCKMATE (Zhao et al., 2023), however at the cost of generality, by proposing solutions only for limited classes of dependency graphs where a chain structure (of potentially complex blocks) can be identified.

The work presented here is at the convergence of these research lines and we propose a computationally efficient, low-overhead solution that can address general graphs. Our framework HIREMATE is based on a **hierarchical decomposition** approach of the computation graph, to find a re-materialization strategy for **any graph of dependencies** between layers. In the case where the graph is too large to be addressed directly by an approach based on Integer Linear Programming, we propose to decompose it into a graph of complex blocks, with potentially several levels in the hierarchy to handle very large graphs. Efficient solutions for different memory budgets are generated for each of the blocks at the bottom of the hierarchy, using different approaches from the literature. Then, we provide a **new** Integer Linear Programming (ILP) formulation to **efficiently recombine** these low-level solutions into candidate solutions for the higher levels of the hierarchy. Furthermore,HIREMATE is fully compatible with the `autograd` mechanism of PyTorch, so that no modification of the code is required to use it. With a single line, the user can automatically control the memory usage of their neural network: `model = HRockmate(model, sample, memory_budget)`.

To achieve this result, we rely on the following main contributions:

- A data-flow graph decomposition algorithm **H-Partition** that builds a hierarchy of blocks of reasonable sizes (to keep an acceptable computational complexity) while minimizing the memory size of the interfaces between the blocks
- A new linear programming solver H-ILP adapted to this hierarchical decomposition.
- A general **framework** for integrating any existing (or future) re-materialization strategy at any level of the hierarchy, and combining their strengths.

The rest of the paper is organized as follows. In Section 2, we review the related work on memory saving strategies for DNN training. HIREMATE is presented in Section 3 which covers graph decomposition, partial problems resolution and global re-materialization strategy reconstruction. Section 4 demonstrates the efficiency of the proposed method on a large number of networks, and provides an evaluation of the computational overhead induced by re-materialization. Finally, concluding remarks are proposed in Section 5.

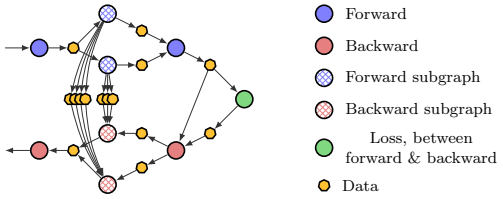

Figure 2. HIREMATE recursively finds schedules for each subgraph. Different schedules correspond to different memory budget constraints and hence have different values for memory/time ratio, peak memory, size of allocated tensors, and execution time.

2. Related Work

We mostly cover the contributions that rely on the exclusive use of re-materialization. Memory footprint of activations might also be reduced via data parallelism (Das et al., 2016; Zhang et al., 2013) (by distributing mini-batches across several computational resources) and offloading (Rhu et al., 2016; Wang et al., 2018) (which offloads and later retrieves activations from GPU memory to CPU memory). Gradient accumulation can also reduce memory usage by splitting a large batch into smaller sub-batches and accumulating gradients across them, but is limited by the low GPU utilization because of smaller sub-batches, and requires that one smaller sub-batch fits in memory. Model

Table 1. Comparison of Different Re-Materialization Strategies in terms of (i) Generality (ability to handle complex models with arbitrary dependencies), (ii) Scalability (ability to handle models with many operations), (iii) Quality (ability to generate a re-materialization strategy with low overhead during training), and Ease of use (in terms of modifications to be made to the original PyTorch model). Scores range from "-" (true limitation) to "++" (excellent).

	GENERALITY	SCALABILITY	QUALITY	EASE OF USE
CHECKMATE (JAIN ET AL., 2020)	++	-	++	0
TW-REMAT (KUMAR ET AL., 2019)	++	++	0	-
MOCCASIN (BARTAN ET AL., 2023)	++	++	+	-
ROCKMATE (ZHAO ET AL., 2023)	+	+	++	+
HIREMATE (PRESENT PAPER)	++	++	++	++

parallelism (Huang et al., 2019; Narayanan et al., 2019), in particular in its pipelined version, can be used to minimize the memory requirements by storing weights across several devices. The optimal combination of all these strategies with re-materialization, discussed by (Beaumont et al., 2021) in the context of sequential models, is left for future work.

TW-Remat (Kumar et al., 2019), based on a tree-width decomposition of the dependency graph, was initially designed for the case where all computational costs and activation sizes have a unitary weight. A greedy heuristic was later proposed to generalize the algorithm to the cases of non-unitary weights, but without guarantee. (Kusumoto et al., 2019) proposed an optimal dynamic programming algorithm restricted to a special class of solutions, where each node is computed at most twice. The XLA framework (xla) contains a re-materialization feature based on a greedy heuristic described in Kumar et al. (2019), and shown to be less efficient than TW-Remat. CHECKMATE (Jain et al., 2020) is based on the solution of an Integer Linear Program (ILP), and computes a generic optimal solution under a set of reasonable assumptions, but the computation cost becomes prohibitive as soon as the number of nodes in the network exceeds 70-90 nodes. Moccasin (Bartan et al., 2023) is a constraint programming based solution we show that HIREMATE does actually outperform Moccasin, both in terms of solution time and the quality of the solution produced.

Other approaches have been proposed to find efficient solutions for limited classes of dependency graphs. (Beaumont et al., 2024 (accepted for publication) relies on dynamic programming in the case of networks whose graph is a sequence of basic operations, covering ResNet-like networks (considering one ResNet block as one operation). This extends in a proven framework the heuristic techniques proposed by (Chen et al., 2016). Then, the case of graphs for which the forward graph is a sequence of complex blocks has been addressed in the ROCKMATE framework in (Zhao et al., 2023), which covers most Transformer-based networks (where one Transformer block is considered as a complex block). However, for architectures with long skip connections (like U-Net (Ronneberger et al., 2015) or encoder-decoder Transformer (Vaswani et al., 2017)), ROCKMATE degenerates to CHECKMATE and inherits its difficulties to handle large data-flow graphs.

Such static rematerialization strategies are very efficient for models with fixed computation graphs, where memory usage can be optimized offline for efficient execution. Nevertheless, they struggle to handle models with dynamic or input-dependent control flow. In contrast, dynamic rematerialization strategies (Kirisame et al., 2020) adapt to runtime behavior by making decisions on-the-fly, allowing them to support a broader range of model architectures. However, this flexibility often comes at the cost of higher runtime overhead and less predictable performance. Notably, dynamic approaches such as POET (Patil et al., 2022) and MegTaiChi (Hu et al., 2022) combine rematerialization with additional mechanisms like paging.

In this paper, we focus specifically on static strategies and we design **an algorithm and a framework to address any kind of network with a static data-flow graph**, with reasonable solving time on networks with up to 2000 nodes.

In the experimental evaluation of Section 4, we compare HIREMATE with the state-of-the-art strategies: CHECKMATE, MOCCASIN, TW-REMAT, and ROCKMATE. Table 1 summarizes the comparison of these re-materialization techniques.

In the experimental evaluation of Section 4, we compare HIREMATE with the state-of-the-art strategies: CHECKMATE, TW-Remat, and ROCKMATE. Table 1 summarizes the comparison of these re-materialization techniques with several metrics. Generality denotes the ability to handle complex models with arbitrary dependencies, scalability denotes the ability to handle models with many operations. Quality measures the ability to generate a re-materialization strategy with low overhead during training, and ease of use is expressed in terms of modifications to be made to the original PyTorch model.

3. HIREMATE

Problem statement We associate each neural network with a *data-flow graph*, where each node represents a tensor-level computation. A *schedule* is a sequence of elementary computations and tensor deletions that performs the forward and backward passes for one training iteration.

Let F_i and B_i denote the forward pass and backward pass associated with layer i and D_i denotes the deletion of the output of F_i. Then, for a model that is a sequence of 3 layers, a schedule corresponding to the standard autodiff training can be written as $F_1F_2F_3B_3B_2B_1$, while $F_1F_2D_1F_3B_3D_3F_1F_2B_2D_2F_1B_1$ is another valid schedule, yielding a smaller peak memory since it does not require storing all intermediate activations simultaneously. **The re-materialization optimization problem** (2) **is to find a schedule whose peak memory is below a given budget and whose execution time is as small as possible.**

Figure 1 describes the main steps of the HIREMATE approach. In the first step, using the graph building tool adapted from (Zhao et al., 2023), **a data flow graph of the input module is obtained**. In the second step, the **H-partition** algorithm (see Section 3.1) **recursively partitions the graph into subgraphs** of manageable sizes. This partitioning is done recursively to ensure that all parent graphs are also of manageable size. Steps 3, 4, and 5 of Figure 1 describe the **H-Solver** algorithm (see Section 3.2), which **builds a training schedule**. Starting at the lowest level of the decomposition, the algorithm computes schedules for each subgraph (Figure 2) with different memory budgets to explore different time-memory tradeoffs, providing several *options* (*i.e* ways to (re)compute operations and store activations during forward and backward passes through the subgraph) for the nodes at higher levels. This procedure continues until the top-level graph is solved (*i.e* schedule is found) for a single memory budget corresponding to the overall memory available for activations. It is important to emphasize that the current implementation of the general scheme described above and in Figure 1 is fully modular. Thus, **HIREMATE allows the injection of custom partitioners as well as solvers at any level of the graph**.

3.1. H-Partition

The goal of HIREMATE's partitioning step is to reduce the size of the problems to be solved without compromising the quality of the overall solution. The result of this step is a decomposition into a hierarchy of subgraphs, where a node at a given level represent an entire subgraph at the level below (see Figure 7 in Appendix). Note that the original graph is partitioned without requiring a topological order; topological sorting is only performed later, within each subgraph, during the schedule generation phase (see Section 3.2). The subgraph sizes are bounded by two main parameters: M^l denotes the maximum number of nodes in a lower-level subgraph, and M^t denotes the maximum number of nodes in the top-level graph. Since it is advantageous to allow a longer solution time for the top-level graph, we use $M^t \geq M^l$. Our partitioning algorithm is a greedy bottom-to-top heuristic described in Algorithm 1. Each iteration has three main steps: forming *candidate* groups, selecting the best candidate according to our evaluation criterion, merging the selected candidate, and updating the candidates. Each step is described below.

Algorithm 1 H-Partition Bottom-to-Top algorithm

Input: data-flow graph G
Result: a recursive partition of G
Parameters: max high-level size M^t, max lower-level size M^l, score parameter α
while G has more than M^t nodes **do**
  $\mathcal{C} \leftarrow \bigcup_{x \in G}$ candidate group containing all nodes between x and $a(x)$
  while $\mathcal{C}$ is non empty and G has more than M^t nodes **do**
    Select candidate C which minimizes s_α (eq. 1)
    Wrap the nodes of C into a group
    Update $\mathcal{C}$
  end while
  Consider all groups as subgraphs
  Update G so that each subgraph is considered as a node
end while
return partitioned graph

Forming candidate groups For each node x in G, we consider $a(x)$, the closest common ancestor of all direct predecessors of x (a common global ancestor is added in case G has multiple entries). We create four candidate groups with all nodes on all paths from $a(x)$ to x, depending on whether x and/or $a(x)$ are included. Any candidate group with more than M^l nodes is discarded.

Selecting the best candidate When selecting the best group among all candidates, our goal is to avoid incurring too much memory pressure when the group is used as a subgraph. The memory pressure depends directly on the size of the input and output values. It also depends, although not as directly, on the length of the schedule during which they will be alive, which we evaluate by the number of compute nodes in the group. We use the following score function $s(C)$ for a candidate C:

$$s_\alpha(C) = \left(\sum_{\substack{x \text{ input or output} \\ \text{value of } C}} \begin{matrix}\text{memory size} \\ \text{of } x\end{matrix} \right) \cdot \left(\begin{matrix}\#\text{ of compute} \\ \text{nodes in } C\end{matrix} \right)^\alpha, \quad (1)$$

where α is a hyperparameter whose default value is 0.5.

Updating the candidates Once the best candidate C is chosen, it becomes a group: all its nodes are considered together from now on. However, it is not yet a subgraph: if it is not too large, it can be merged with other groups later in the same phase to reduce the number of groups. The remaining candidates are updated: in each candidate C' whose intersection with C is not empty, we add all other vertices of C to ensure that all vertices of C are kept together. If this union contains more than M^l nodes, this candidate C' is no longer acceptable and is removed.

Section A in Appendix contains a proof that this procedure always leads to a valid decomposition, in the sense that no subgraph contains a directed cycle. After partitioning, HIREMATE identifies subgraphs that correspond to the execution of the same piece of code to avoid solving the same optimization problem multiple times.

3.2. H-Solver

3.2.1. SOLVING FRAMEWORK

The idea of hierarchical re-materialization is that re-materialization schedules can be computed for each subgraph independently, with multiple possible memory budgets. The resulting re-materialization schedules for a given subgraph are called *options*, and each corresponds to a different tradeoff between computation time and required memory. **Once all subgraphs at a given level of hierarchy have been resolved, a schedule for the upper level can be computed with our proposed H-ILP**. H-ILP is an Integer Linear Program (ILP) formulation that provides an optimal schedule within a given memory budget. It can be used on a subgraph of arbitrary structure, but using it on large subgraphs can lead to unreasonably long solution times. Therefore, it is only applied to subgraphs with a sufficiently small number of nodes. This algorithm is inspired by RK-CHECKMATE (Zhao et al., 2023) and CHECKMATE (Jain et al., 2020). The general idea of extending an ILP-based approach to a graph of arbitrary size and structure, which is hierarchically decomposed into subgraphs of manageable size, is one of the major contributions of the present work and is described in detail in Section 3.2.2.

For significantly large graphs, partitioning with only two levels would result in the top-level graph still being too large to be solved with this technique. Our hierarchical approach makes it possible to introduce more than two levels into the hierarchy and compute schedules for the levels from bottom to top, handling graphs of arbitrary size while keeping each subproblem of manageable size.

The H-ILP solver we propose is very generic and efficient, and it provides very good quality solutions on all types of neural networks. **Thanks to the genericity and extensibility of the HIREMATE framework, it allows combining different solvers**, which further improves the quality of the solutions. All solvers can be used at any level, but those not acting hierarchically ignore the underlying partition. Each solver comes with an *applicability* criterion to decide whether it is suited for a given graph. For each subgraph, all applicable solvers are used to produce schedules.

The HIREMATE framework currently includes two additional algorithms. First, H-TWREMAT is a wrapper around the TW-REMAT implementation (Shepperd, 2021) of a heuristic based on a **treewidth decomposition** approach (Kumar et al., 2019). This wrapper and the HIREMATE framework allow this heuristic to be used with PyTorch where previously it was only available for TensorFlow. Second, RK-ROTOR is the **dynamic programming** algorithm from (Zhao et al., 2023), which also provides very good schedules in the case of (forward) graphs consisting of a sequence of possibly complex subgraphs. It is therefore limited in genericity, but its computational time is low. The RK-ROTOR solver is only activated when we detect that the graph can be decomposed into a sequence.

3.2.2. HIERARCHICAL ILP FORMULATION

Consider an arbitrary graph H, where each computation node represents a subgraph (Figure 2), and where dependencies are carried by *data nodes* representing values that can be stored in memory. The H-ILP formulation computes the minimum runtime schedule whose peak memory remains below a given memory budget. The computation nodes are numbered in topological order.

In the linear programming formulation of H-ILP, the schedule is divided into *phases*, and the goal of phase t is to compute node t for the first time. For the sake of brevity, we describe here only the main ideas that allow H-ILP to be used in a hierarchical setting, but refer the reader to the Section B of Appendix for a complete description of the H-ILP formulation.

Compute options and phantom nodes The innovation of H-ILP is that each compute node can represent a subgraph of the original graph. Such a compute node can be computed with one of several *options*. Each of these options represents a possible schedule for the forward and backward phases of the associated subgraph. There is a tight coupling between the forward computation and its corresponding backward computations, and each backward computation should be performed with the same option as its corresponding forward computation. The H-ILP formulation contains additional variables and constraints to specify which option is used for each computation node within each phase.

In H-ILP, we also introduce an explicit representation of the data stored in memory between a forward computation and its corresponding backward computation. We call them

phantom nodes, and we update the formulation by considering them as special data nodes, with two specificities. First, a phantom node is always created by its forward computation, always deleted after its backward computation, and is not used by any other computation node. In the formulation, we take advantage of this by not including additional variables expressing whether the phantom node is deleted or not. Second, the values stored in a phantom node (and thus the associated memory size) depend on the option used for the forward and backward computations. For this reason, the formulation includes additional variables that specify which option of each phantom node is in memory at each stage.

Objective Let $R^t_{i,o}$ equals 1 if node i is computed with option o during phase t and 0 otherwise. Then the objective is to minimize the total execution time expressed as

$$\min \sum_{i,t,o} R^t_{i,o} * \text{time of computing option } o \text{ for node } i \quad (2)$$

subject to constraints on auxiliary variables specified in Section B.3 of Appendix.

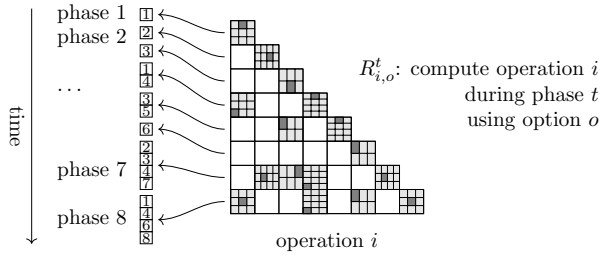

Figure 3. Example of the solution. $R^t_{i,o}$ equals 1 if node i is computed with option o during phase t.

Correction terms for memory usage In (Zhao et al., 2023), to achieve good performance, the input values of a block are deleted as soon as they are no longer needed, instead of being kept until the end of the block. This is possible due to the sequential structure, where each block is the only user of its input values. In the more general context of this paper, a subgraph may share its input values with other subgraphs. To achieve good performance, H-ILP allows subschedules to delete their input values and the gradients associated to their outputs; however, if another subgraph needs these values, the higher-level algorithm adjusts the subschedules. This decision has an impact on the memory consumption of the corresponding subgraphs.

We have added constraints to the formulation to ensure that the memory peak is correctly evaluated in all cases. These constraints are introduced only when solving the top-level graph, since they make the ILP harder to solve, and only the top-level graph solution is required to fulfill accurately the GPU memory bound. For each computation node k, the number of these constraints for node k is bounded in the worst case by $\min(l_{k,o}, 2^{\text{degree of node } k})$, where $l_{k,o}$ is the number of operations of the schedule associated to option o for node k. In practice, this number of constraints remains small enough so that the H-ILP formulation can be solved in reasonable time.

Schedule selection The number of binary variables in the H-ILP formulation depends linearly on the total number of options of all nodes. To avoid wasting resources when several very similar options are available for a given node, we include in H-ILP a hyperparameter N_o that imposes a limit on the total number of options. To stay within this limit, we may need to select only a few options from all the schedules generated at the lower level. To do so, we split N_o to assign a number of options to each node proportional to the number of basic operations within the associated subgraph. Then, we greedily select the schedules whose memory peaks are farther apart from each other: starting with the schedule with the highest memory peak, then the one with the lowest memory peak, then the one closest to the middle, and so on.

3.3. Complexity Analysis

The complexity of using the H-ILP solver depends mostly on the number N of nodes in the graph. Obtaining the dataflow graph with RK-GB has a complexity $O(N)$ and is very fast in practice. With our current implementation, recursively partitioning the graph with H-partition has a complexity of $O(N^2 \log N)$. This step is also fast for graph sizes up to $N = 1000$, but handling very large graphs would require more work on the graph algorithms: recursively partitioning a graph of size $N = 10^5$ takes 2 hours on an Intel Xeon Gold processor.

The most time-consuming step is the computation of re-materialization schedules with H-ILP. Thanks to our hierarchical approach, this step actually has linear complexity. In fact, the hierarchical decomposition has a logarithmic depth, and the total number of subgraphs is $O(N/M^t)$. Solving a subgraph of size M^l for a number O of budget options only requires solving O Integer Linear Programs, whose sizes and solving times do not depend on N. Moreover, all ILPs at the same level are completely independent and can be run in parallel. As detailed in the Appendix, even without parallelization, our current implementation is able to handle the largest versions of neural networks used in practice: the forward-backward graphs of GPT with 96 layers and a Transformer with 36 encoders and decoders have 2500 nodes and are solved in 15 and 150 minutes, respectively.

Figure 4. Experiments for varying graph size: (top) iteration time (bottom) solving time

4. Experimental Evaluation

Experimental settings The HIREMATE framework is designed to be used for end-to-end training. Since re-materialization guarantees that all computed values (including the gradients) are the same as without any recomputation, we focus in our experiments on measuring peak memory usage and iteration time for a single training iteration. We perform a warm-up phase consisting of five initial runs; the subsequent ten runs are used to evaluate the peak memory and computation time, providing reliable estimates of performance. Measurements show that the standard error of the iteration time is at least two orders of magnitude smaller than the mean. As a result, error bars are not shown in the plots. The experiments were performed on an NVIDIA Quadro RTX8000 GPU with 48 GB of memory and an NVIDIA V100 GPU with 16 GB of memory, using PyTorch 2.0.1, CUDA 11.6, and Gurobi 9.5.0. We intentionally report performance on settings where the experimental platform can run the original model, so that we can compare our results with the training time obtained with regular PyTorch Autodiff. All experiments can be scaled up by increasing image or batch size, to a point where training requires using HIREMATE. Additional experiments including an ablation study, varying batch sizes and sequence lengths, as well as a dozen different architectures are available in the Appendix.

Efficiency In Figure 4 we compare the solving time and performance of ROCKMATE, CHECKMATE, MOCCASIN, and H-ILP. Since solving an ILP has exponential complexity, it is important to limit the size of the problem. A clear disadvantage of ROCKMATE is that it needs to see the graph as a sequence of blocks, which for general neural networks means that some blocks are very large, leading to unacceptable solving times for graphs that do not follow this structure. This is typically the case in the encoder-decoder transformer (Vaswani et al., 2017), as shown on the bottom of Figure 4. Specifically, ROCKMATE takes 15 hours to obtain a solution for a 4-layer encoder-decoder structure transformer, while H-ILP achieves better results within 12 minutes. Note that H-ILP is run once before the entire model training, so 12 minutes is clearly acceptable. By controlling the total number of nodes in each graph with the H-partition algorithm, H-ILP is able to efficiently limit the solving time without compromising solution quality. It is important to note that though MOCCASIN claims to beat CHECKMATE, it does not provide an optimal solution in general, because the number of rematerializations is bounded in practice (to 2, to limit complexity), and the time allocated to the constraint programming solver is also bounded (to 1h, because the time to reach a solution can be arbitrarily long).

Performance Figure 5 shows that H-ILP consistently outperforms TW-REMAT in iteration time for a given budget. On the encoder-decoder (Figure 5(c)), TW-REMAT is able to find schedules for smaller memory budgets than H-ILP, but for UNet the situation is the reverse. The HIREMATE algorithm refers to the complete framework, including the RK-ROTOR and TW-REMAT solvers. By integrating TW-REMAT, HIREMATE overcomes this limitation of H-ILP and provides efficient solutions over the entire range of memory budgets. Figures 5(a) and 5(b) show that while H-ILP may perform slightly worse than ROCKMATE, H-ILP achieves similar performance to ROCKMATE, a baseline approach specifically tailored for such architectures.

Overall, HIREMATE acts as a general solution that includes H-ILP, ROTOR, ROCKMATE, and CHECKMATE. Our experimental results consistently show that HIREMATE performs on par with these baseline algorithms, demonstrating its versatility and effectiveness in optimizing memory utilization for a wide range of network architectures.

Controlling Sub-Optimality With our hierarchical approach, the H-ILP solver does not compute an optimal solution to the re-materialization problem. We compare our results with RK-CHECKMATE, which provides an optimal solution for a given topological order of operations. The selected results in Table 2 show that H-ILP achieves performance close to this optimal solver on a network size where it remains tractable to compute. In another experiment, we investigate the impact of hierarchy depth in the H-partition by restricting the size of each subgraph, thereby increas-

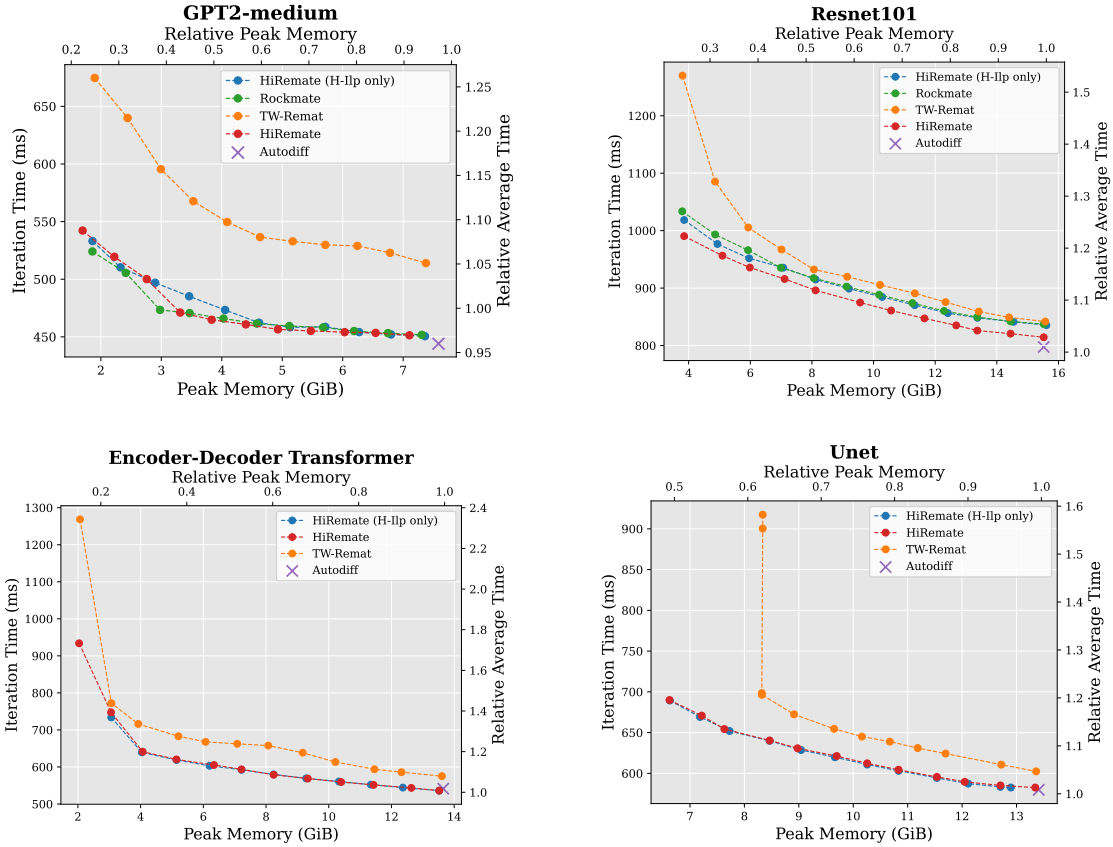

Figure 5. Experiments on different network architectures: (top) sequential-like neural networks (bottom) non-sequential neural networks, which ROCKMATE cannot solve efficiently.

ing the depth. As shown in Figure 6, the solution quality achieved by H-ILP shows only a slight degradation as the hierarchy becomes deeper. This indicates that HIREMATE effectively preserves performance despite deeper partitioning. It is important to note, however, that increasing the depth significantly affects the complexity and tractability of the problem as shown in Section 3.3, making this result particularly notable: H-ILP manages to avoid quality degradation in settings where solving the problem becomes considerably more challenging. The HIREMATE framework allows the user to use several trade-offs between solution quality and solving time, by adjusting the partition granularity, the number O of memory budget levels used in lower-level subgraphs, or the number N_o of options retained when solving one subgraph.

5. Discussion and Conclusion

This paper introduces the HIREMATE framework, which offers both theoretical and practical advances in re-materialization for PyTorch models. HIREMATE provides a solution that can find very effective solutions in terms of overhead during training, comparable to those of the literature for problems of small size or with a particular graph structure, but without these limitations. On the practical side, HIREMATE integrates seamlessly with PyTorch, improving the memory-time tradeoff and efficiency, and is fully compatible with PyTorch Autograd. The theoretical contributions include a hierarchical approach and an original linear programming formulation H-ILP. Although HIREMATE focuses on computational graphs with primitive operations, the hierarchical approach with linear complexity is potentially applicable to other intensive tasks targeting

Table 2. Comparison of H-ILP and RK-CHECKMATE overhead in terms of iteration time

Network	Budget (GB)	Checkmate	H-ILP
UNet	7.1	5.32%	5.44%
2-layer Tformer	7.0	5.32%	5.34%
MLP-Mixer	6.0	2.47%	2.44%
5-layer GPT2	1.25	6.02%	9.24%

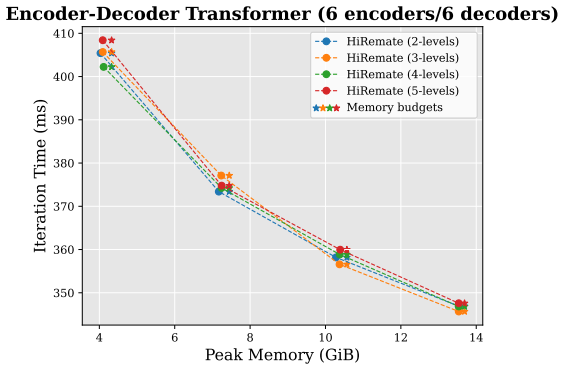

Figure 6. Experiments on 6-layer encoder-decoder Transformer with different depth of hierarchical structure.

graphs resulting from tiling. The framework also incorporates previous approaches, ensuring state-of-the-art results. The remaining limitation of HIREMATE is that it is not adapted to dynamic neural network architectures, where the the structure of the computational graph changes based on runtime inputs. However, HIREMATE can be extended to support mildly dynamic graphs by unrolling fixed-length control flow, precomputing schedules for a small number of known variants, or applying scheduling locally to submodules with static behavior. A critical point is that the code is fully modular, and it is possible for external contributors to introduce a new graph partitioner or solver at any level of the hierarchical decomposition. We hope that this modularity will stimulate research and further improvement of HIREMATE. Future research may include exploring how to integrate offloading into the optimization problem to reduce the need for recomputation, and how to optimally combine rematerialization with pipelined model parallelism. Advances in these areas hold promise for improving the performance and efficiency of deep learning systems.

Acknowledgments

The research has been partially supported by the EUPEX (European Pilot for Exascale) project, which received funding from the European High-Performance Computing Joint Undertaking (JU) under grant agreement No 101033975.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

XLA. http://www.tensorflow.org/xla.

Bartan, B., Li, H., Teague, H., Lott, C., and Dilkina, B. Moccasin: Efficient tensor rematerialization for neural networks. In *International Conference on Machine Learning*, pp. 1826–1837. PMLR, 2023.

Beaumont, O., Eyraud-Dubois, L., Hermann, J., Joly, A., and Shilova, A. Rotor, 2019. URL https://gitlab.inria.fr/hiepacs/rotor.

Beaumont, O., Eyraud-Dubois, L., and Shilova, A. Efficient combination of rematerialization and offloading for training dnns. *Advances in Neural Information Processing Systems*, 34:23844–23857, 2021.

Beaumont, O., Eyraud-Dubois, L., Hermann, J., Joly, A., and Shilova, A. Optimal checkpointing for heterogeneous chains: how to train deep neural networks with limited memory. *ACM Transactions on Mathematical Software (TOMS)*, 2024 (accepted for publication). URL https://arxiv.org/abs/1911.13214.

Chen, T., Xu, B., Zhang, C., and Guestrin, C. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

Das, D., Avancha, S., Mudigere, D., Vaidynathan, K., Sridharan, S., Kalamkar, D., Kaul, B., and Dubey, P. Distributed deep learning using synchronous stochastic gradient descent. *arXiv preprint arXiv:1602.06709*, 2016.

Hu, Z., Xiao, J., Deng, Z., Li, M., Zhang, K., Zhang, X., Meng, K., Sun, N., and Tan, G. Megtaichi: Dynamic tensor-based memory management optimization for dnn training. In *Proceedings of the 36th ACM International Conference on Supercomputing*, pp. 1–13, 2022.

Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M., Lee, H., Ngiam, J., Le, Q. V., Wu, Y., et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems*, pp. 103–112, 2019.

Jain, P., Jain, A., Nrusimha, A., Gholami, A., Abbeel, P., Gonzalez, J., Keutzer, K., and Stoica, I. Checkmate: Breaking the memory wall with optimal tensor rematerialization. *Proceedings of Machine Learning and Systems*, 2:497–511, 2020.

Kirisame, M., Lyubomirsky, S., Haan, A., Brennan, J., He, M., Roesch, J., Chen, T., and Tatlock, Z. Dynamic tensor rematerialization. *arXiv preprint arXiv:2006.09616*, 2020.

Kumar, R., Purohit, M., Svitkina, Z., Vee, E., and Wang, J. Efficient rematerialization for deep networks. *Advances in Neural Information Processing Systems*, 32, 2019.

Kusumoto, M., Inoue, T., Watanabe, G., Akiba, T., and Koyama, M. A graph theoretic framework of recomputation algorithms for memory-efficient backpropagation. *Advances in Neural Information Processing Systems*, 32, 2019.

Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020.

Narayanan, D., Harlap, A., Phanishayee, A., Seshadri, V., Devanur, N. R., Ganger, G. R., Gibbons, P. B., and Zaharia, M. PipeDream: generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pp. 1–15, 2019.

Naumann, U. Call tree reversal is np-complete. In *Advances in automatic differentiation*, pp. 13–22. Springer, 2008.

Patil, S. G., Jain, P., Dutta, P., Stoica, I., and Gonzalez, J. Poet: Training neural networks on tiny devices with integrated rematerialization and paging. In *International Conference on Machine Learning*, pp. 17573–17583. PMLR, 2022.

Rahman, M. A., Ross, Z. E., and Azizzadenesheli, K. U-no: U-shaped neural operators. *arXiv preprint arXiv:2204.11127*, 2022.

Rhu, M., Gimelshein, N., Clemons, J., Zulfiqar, A., and Keckler, S. W. vdnn: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *The 49th Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 18. IEEE Press, 2016.

Ronneberger, O., Fischer, P., and Brox, T. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18*, pp. 234–241. Springer, 2015.

Shepperd, N. Fine tuning on custom datasets, 2021. URL https://github.com/nshepperd/gpt-2/tree/finetuning/twremat.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Wang, L., Ye, J., Zhao, Y., Wu, W., Li, A., Song, S. L., Xu, Z., and Kraska, T. Superneurons: Dynamic gpu memory management for training deep neural networks. *SIGPLAN Not.*, 53(1):41–53, February 2018. ISSN 0362-1340. doi: 10.1145/3200691.3178491.

Wen, G., Li, Z., Azizzadenesheli, K., Anandkumar, A., and Benson, S. M. U-fno—an enhanced fourier neural operator-based deep-learning model for multiphase flow. *Advances in Water Resources*, 163:104180, 2022.

Wu, Z., Shen, C., and Van Den Hengel, A. Wider or deeper: Revisiting the resnet model for visual recognition. *Pattern Recognition*, 90:119–133, 2019.

Zhang, S., Zhang, C., You, Z., Zheng, R., and Xu, B. Asynchronous stochastic gradient descent for dnn training. In *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, pp. 6660–6663. IEEE, 2013.

Zhao, X., Le Hellard, T., Eyraud-Dubois, L., Gusak, J., and Beaumont, O. Rockmate: an Efficient, Fast, Automatic and Generic Tool for Re-materialization in PyTorch. In *ICML 2023*, Honolulu (HI), United States, July 2023. URL https://hal.science/hal-04095305.